南昌航空大學

毕业设计（论文）

题　　目：基于机器学习方法的链路预测

学　　院：信息工程学院

专业名称：计算机科学与技术

班级学号：21045335

学生姓名：周彤

指导教师：张胜、程世明

二零二五 年 六 月

南昌航空大學

毕业设计（论文）任务书

I、毕业设计(论文)题目：

基于机器学习方法的链路预测

II、毕业设计(论文)使用的原始资料(数据)及设计技术要求：

原始资料：指导老师给的论文、网上下载的资料。

设计要求：

1. 搜集面向行业应用的数据集；
2. 了解链路预测的基本概念，掌握相关链路预测算法的基本原理；
3. 选择并实现相应的链路预测算法；
4. 掌握现代编程工具，根据实验结果，比较、分析上述链路预测算法；
5. 讨论问题解决方案对社会、健康、安全、法律以及文化的影响；
6. 提交与毕设课题相关的英文原稿及翻译。

III、毕业设计(论文)工作内容及完成时间：

1. 第 1 周 — 第 3 周　收集资料，了解链路预测的基本概念及常见的链路预测算法，完成开题报告；完成 10000 以上印刷符号的与本课题相关的英文技术资料的翻译。
2. 第 4 周 — 第 12 周　设计并实现经典的链路预测算法；对各个链路预测算法进行编码、测试。
3. 第 13 周 — 第 15 周　比较、分析各链路预测算法的优劣，撰写毕业论
4. 第 16 周　修改论文，准备毕业答辩。

Ⅳ、主要参考资料：

[1] 陈亮，赵英，史晟辉，等. 基于超图神经网络的链路预测方法[J/OL]. 计算机工程，1-12[2024-12-28].

[2] 毕祎琳，焦鑫善，万书言，等. 链路预测的若干基础问题探讨[J]. 电子科技大学学报，2024,53(5):792-800.

[3] 袁源，郭进利. 基于复杂网络上监督学习方法的研究综述[J]. 运筹与管理,2022,31(12):234-239.

[4] 吴金红，黄彩云，李问秋，等. 融合深度学习和链路预测的交叉性研究主题预测研究[J]. 情报杂志，2022,41(12):57-63.

[5] 黄璐，朱一鹤，张嶷. 基于加权网络链路预测的新兴技术主题识别研究[J].情报学报，2019,38(04):335-341.

[6] Zhang, DK; Yin, J; YU, PS; Link Prediction with Contextualized Self-Supervision[J]. IEEE Transactions on Knowledge and Data Engineering, 2023, 35(7), 7138 – 7151.

[7] Cai, L; LI, JD; WANG, J; et al. Line Graph Neural Networks for Link Prediction[J].IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022, 44(9), 5103 – 5113.

信息工程 **学院** 计算机科学与技术 **专业类** 210453 **班**

学生（签名）：

日期： **自** 2025 **年** 01 **月** 06 **日至** 2025 **年** 06 **月** 01 **日**

指导教师（签名）：

助理指导教师(并指出所负责的部分)：

计算机应用技术 **系（室）主任（签名）：**

附注:任务书应该附在已完成的毕业设计说明书首页。

南昌航空大學

学士学位论文原创性声明

本人声明，所呈交的学位论文是本人在指导教师的指导下独立完成的研究成果。除了文中特别加以标注引用的内容外，本论文不包含法律意义上已属于他人的任何形式的研究成果，也不包含本人已用于其他学位申请的论文或成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式表明。本人完全意识到本声明的法律后果由本人承担。

作者签名：　　日期：

导师签名：　　日期：

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权南昌航空大学可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

作者签名：　　日期：

导师签名：　　日期：

摘　要

随着互联网和社交网络的快速发展，网络数据的规模呈指数级增长，各类复杂网络包括但不限于社交网络、生物网络、信息网络、交通网络、技术网络、经济网络等快速发展。基于此背景，链路预测作为复杂网络分析的关键问题之一，旨在预测网络中当前尚未连接的但可能存在的边，面临着预测精度有待提高等问题，因此常运用机器学习方法解决链路预测面临的问题。

本文主要对比了逻辑回归（Logistic regression, LR）算法和逻辑回归 LightGBM 堆叠链路预测（Logistic-regression LightGBM Stacking Link Prediction method，LLSLP）算法。其中 LR 算法是经典机器学习算法。LLSLP 算法从特征提取、模型构建和算法优化等方面着手。将网络的链路预测视为一个二分类问题，考虑每两个节点间的 8 个相似性指标，将这些相似性指标视为网络中任何两个节点间的特征；为了提高预测准确性和泛化性，选择差异较大的模型作为基模型。代码中采用逻辑回归和 2 种 LightGBM 作为基模型进行集成，二者的差异有助于提升整体性能，并采用 5 折交叉验证的方法进行基模型的训练和预测。然后将逻辑回归和 LightGBM 预测的链接存在与不存在的概率作为特征输入到 LightGBM 元模型中，实现了特征的重新利用，以提升模型的预测性能。

在五个网络上进行了对比实验，将 LLSLP 算法和 LR 算法在评估指标方面包括 AUC、准确率、精确率、召回率和 F1 分数进行评估比较，实验表明该算法相较于 LR 算法具有一定的优化作用。

关键词：链路预测；机器学习；逻辑回归；Stacking

Abstract

With the rapid development of the Internet and social networks, the scale of network data is growing exponentially, and various types of complex networks including but not limited to social networks, biological networks, information networks, transportation networks, technological networks, economic networks, etc. are developing rapidly. Based on this background, link prediction, as one of the key problems in complex network analysis, aims at predicting the currently unconnected but possible edges in the network and faces problems such as prediction accuracy to be improved, so machine learning methods are often applied to solve the problems faced by link prediction.

This dissertation mainly compares the Logistic regression (LR) algorithm and the Logistic-regression LightGBM Stacking Link Prediction method (LLSLP) algorithm. Among them, LR algorithm is the classical machine learning algorithm. LLSLP algorithm starts from feature extraction, model construction and algorithm optimization. The link prediction of the network is regarded as a binary classification problem, and 8 similarity metrics between every two nodes are considered, and these similarity metrics are regarded as the features between any two nodes in the network; in order to improve the prediction accuracy and generalization, the model with large differences is chosen as the base model. The code uses logistic regression and 2 LightGBMs as base models for integration, and the differences between the two help to improve the overall performance, and a 5-fold cross-validation method was used for training and prediction of the base model. The probabilities of the presence and absence of links predicted by logistic regression and LightGBM are then input as features into the LightGBM metamodel, enabling the reuse of features to improve the predictive performance of the model.

Comparison experiments are conducted on five networks to evaluate and compare the LLSLP algorithm with the LR algorithm in terms of evaluation metrics including AUC, accuracy, precision, recall, and F1 scores, and the experiments show that the algorithm is optimized compared to the LR algorithm.

Keywords: Link Prediction; Machine Learning; Logistic Regression; Stacking

目　　录

第 1 章 引言 1
1.1 研究背景及意义 1
1.2 国内外发展现状与发展趋势 2
1.2.1 国内研究概况 2
1.2.2 国外研究概况 2
1.2.3 链路预测发展历程 2
1.2.4 链路预测发展趋势 3
1.3 研究内容及结构 4
第 2 章 相关基础知识 5
2.1 复杂网络及图论基础 5
2.1.1 图的基本分类 5
2.1.2 邻接矩阵 6
2.2 网络基本拓扑性质 7
2.2.1 网络大小与密度 7
2.2.2 度与度分布 7
2.2.3 网络的平均路径长度与直径 8
2.2.4 聚类系数 8
2.2.5 同配系数 9
2.3 链路预测基础 9
2.3.1 链路预测问题描述 9
2.3.2 数据集的划分 10
2.3.3 性能评价指标 10
2.4 传统链路预测方法 12
2.4.1 基于相似性的方法 12
2.4.2 基于路径的方法 13
2.5 基于机器学习的链路预测算法 14
2.5.1 逻辑回归算法 14
2.5.2 LightGBM 算法 14
第 3 章 算法原理与代码实现 16
3.1 算法分析与选择 16

3.2 逻辑回归算法 16
3.2.1 算法设计思想 16
3.2.2 代码实现 17
3.3 LLSLP 算法 19
3.3.1 算法设计思想 19
3.3.2 Stacking 集成学习模型架构 20
3.3.3 数据生成与预处理 24
3.3.4 模型预测与评估 27
第 4 章 实验结果与分析 29
4.1 实验环境 29
4.2 数据集 29
4.3 实验结果与分析 29
4.3.1 实验结果 29
4.3.2 实验结果分析 29
第 5 章 演示软件的设计与实现 33
5.1 开发平台 33
5.1.1 前端开发环境 33
5.1.2 后端开发环境 33
5.2 系统设计 33
5.2.1 架构设计 33
5.2.2 系统介绍 33
5.2.3 系统功能模块设计 34
5.3 系统实现 34
5.3.1 系统介绍模块实现 34
5.3.2 链路预测模块实现 35
5.3.3 模型对比模块 37
5.3.4 错误消息本地化 39
5.3.5 请求和响应拦截器 40
第 6 章 结论与展望 42
6.1 研究工作总结 42
6.2 研究不足与展望 42

参考文献 43
致　　谢 45

第 1 章 引言

1.1 研究背景及意义

在网络与社交网络迅猛发展的时代，网络数据量级是指数级的，复杂网络[1]是一门诠释各类网络现象和网络复杂性的一个新兴学科。它已成为重要的数学框架，可以用来描述众多现实对象，涵盖社交网络[2]、生物网络[3]、信息网络[4]、交通网络[5]、技术网络[6]、经济网络[7]等多个领域。

在复杂网络中，节点和边构成了复杂的图结构。图分析的关键任务是链路预测，可以用来预测网络中还没连接的节点之间未来可能形成的边。随着各个领域，例如社交网络、推荐系统、生物信息学、交通系统等的快速发展，链路预测的重要性愈发显著。比如，在社交网络里，链路预测能够为用户推荐好友或潜在的兴趣群体；生物信息学中，可用于预测蛋白质之间的相互作用，助力理解生物系统的复杂性；推荐系统里，能帮助预测用户与商品间的潜在联系，提升推荐的精准度；交通系统方面，链路预测方法还可对航空网络进行重构。

从理论意义上看，链路预测的关键点在于打通了网络科学与机器学习的学科边界，一方面将网络科学的图顶点与机器学习的模式学习相结合，形成跨学科的方法体系；另一方面，通过真实世界网络的建模和求证的结合，推动了算法设计与现实的结合。

传统的链路预测方法大多是依赖于手工设计的特征，像 Common Neighbor (CN)[8] 指标、Jaccard (JC)[9] 指标、Adamic - Adar (AA)[10] 指标等。这些算法简单直观，但在处理大规模复杂网络时难以捕捉网络中的非线性关系。而基于机器学习的链路预测方法能够较好的优化这一弊端，它能够自动从数据中学习特征，并借助一些复杂模型进行预测，显著提高了预测的准确性。量化研究显示，在多个公开数据集上，基于机器学习的链路预测方法表现优异。

研究基于机器学习的链路预测有着重要的理论意义和广泛的应用价值。在理论上，链路预测是图分析的经典问题，基于机器学习的链路预测研究能够推动图分析理论的进展，并能为其他图分析任务，如图分类、节点分类等，提供新的思路与手段。

在此基础上，为了实现复杂网络链路预测，本文实现了逻辑回归[11]（Logistic Regression，LR）算法和 LLSLP 算法。其中 LR 算法是经典的机器学习算法，而 LLSLP 算法则将链路预测问题转化为二分类问题，综合考虑节点间的多个相似性

指标，并将其作为网络中任意两个节点的特征。同时，选择 Logistic Regression 和 LightGBM[12] 作为基础模型，引入 Stacking 思想，对基础模型的预测结果进行再学习，从而提升链路预测的准确度。LLSLP 方法通过整合弱学习器的偏置或方差，构建出性能更优的强学习器（即“集成模型”），以利用多个简单模型的优势构建一个整体性能更强的复合模型。

1.2 国内外发展现状与发展趋势

1.2.1 国内研究概况

国内对链路预测研究起步虽然相对较晚，但发展迅速，近几年来取得了一系列重要成果。在理论研究方面，国内学者研究了链路预测模型、理论方法、应用等方面内容，提出了许多创新性的理论和方法。例如，基于随机游走和矩阵分解的链路预测模型解决了传统链路预测方法不能处理大规模网络的问题；针对异质信息网络链路预测问题，提出了基于元路径的链路预测方法等。

在数据集建设方面，国内学者构建了多个公开的链路预测数据集，为链路预测研究提供了数据支持，包括 AMiner 学术社交网络数据集、DBLP 计算机科学文献数据集等。

1.2.2 国外研究概况

国外对于链路预测研究较早，发展比较成熟，理论研究及应用研究均有较多研究成果。

在理论研究方面，国外学者在链路预测模型、算法和应用等方面进行了深入研究，提出了许多经典的理论和方法。例如，Liben-Nowell 和 Kleinberg 提出了基于节点相似度的链路预测方法[13]；Newman 提出了基于随机图的链路预测方法[14]；Menon 和 Elkan 提出了基于矩阵分解的链路预测方法等[15]。

在数据集建设方面，国外研究人员创建了多个公开的链路预测数据集，用于支持链路预测的研究。例如，Facebook 社交网络数据集、Twitter 社交网络数据集、Wikipedia 超链接网络数据集等。

1.2.3 链路预测发展历程

链路预测是网络科学的重要研究方向，在最开始的探索阶段，研究主要集中在社会学和数学领域，主要是为了理解和描述社会网络的结构特征，20 世纪 70 年代数学家 Erdős 和 Rényi 提出了随机图模型，为网络科学研究奠定了基础，20 世

纪 80 年代社会学家 Granovetter 提出了“弱连接”理论，强调了弱连接在社会网络中的重要性。

20 世纪 90 年代到 21 世纪初的研究主要集中在计算机科学领域，主要的目标是开发有效的链路预测算法。90 年代提出相似度的链路预测方法，例如共同邻居、Jaccard 系数、Adamic-Adar 指数等。21 世纪初基于随机游走和矩阵分解的链路预测，例如 Katz[16]指数、局部路径指数和奇异值分解等。

在 21 世纪初到 2010 年代中期的研究以机器学习和数据挖掘为主，旨在利用机器学习算法提高链路预测的准确率。21 世纪初提出基于机器学习的链路预测，例如逻辑回归、支持向量机和随机森林等。2010 年代提出了基于特征工程的链路预测方法，例如利用节点属性、边属性、网络结构特征等进行链路预测。

2010 中后期至今主要基于深度学习，旨在通过深度学习网络，自动学习网络特征表示增强网络链路预测效果。2010 中后期至今提出了基于机器学习的链路预测方法，例如 Deepwalk[17]、Node2Vec[18]等，以及基于图神经网络的链路预测方法，例如图卷积网络、图注意力网络等。

1.2.4 链路预测发展趋势

随着互联网海量信息的涌现和机器学习方法的进步，基于机器学习的链路预测方法会产生如下趋势。（1）从浅层模型到深度模型。传统的机器学习模型，如逻辑回归、支持向量机等，不适用于网络中的复杂非线性关系。深度模型如图神经网络[19]（GNN）可以利用多层非线性变换学习到更复杂、更高阶的网络结构信息，在链路预测任务中表现优秀。（2）从单一模型到混合模型。单一机器学习模型的链路预测性能通常很难挖掘网络全部特征，考虑将不同的机器学习模型进行组合，如特征工程模型和表示学习模型，取长补短，来提高网络链路预测精度。（3）从静态模型到动态模型。传统的链路预测模型主要针对静态网络，但是实际上面对的是动态变化的网络，开发能够处理动态网络的链路预测模型，例如基于时间序列分析的模型、基于强化学习的模型等，是未来的重要研究方向。（4）从同质网络到异质网络。传统的链路预测模型大多是针对于同质网络，即网络中所有的节点与边属于同一类。现实生活中的网络大多为异质网络，比如社交网络中具有各种用户、群、帖等类型的节点，研究能处理异质网络的链路预测模型也是未来一大热点。

1.3 研究内容及结构

本文旨在探讨基于机器学习方法的链路预测，实现 LR 算法和 LLSLP 算法，在 5 个真实网络中进行对比，分析模型的性能，主要是 AUC、准确率、精确率、召回率和 F1 分指标。各章节安排如下：

第一章为引言，旨在探讨链路预测的研究背景、重要性以及国内外研究现状，研究内容及结构。

第二章为相关基础知识，包括复杂网络与链路预测基础，然后介绍链路预测方法。

第三章介绍 LR 算法和 LLSLP 算法实现原理与代码实现。

第四章展示实验结果并对结果进行分析，将 LLSLP 算法和 LR 算法进行比较，直观展示不同算法的性能差异。

第五章对演示软件的设计进行介绍。

第六章为总结与展望。

第 2 章 相关基础知识

2.1 复杂网络及图论基础

2.1.1 图的基本分类

图论中的网络可分为四大类。第一种按方向性，无向图没有方向（如社交网络好友关系），有向图有方向（如推特关注网络）。第二种按权重：无权图边没有权重信息，加权图（如交通网络）边含权重信息。第三种按时变性：静态图拓扑固定，动态图（如通信网络）随时间演化。第四种按异质性：同质图节点类型单一，异质图有多个节点类型（如学术网络含作者、论文等多类型节点）。在链路预测中，特征提取的方法需要根据网络类型来选择，如针对有向网络需区分入度/出度特征。

本文主要根据边的方向和权重，根据其边是否有方向和权值分为四类：加权有向图，无权有向图，加权无向图和无权无向图。图 2.1 展示了四类图之间的关系。

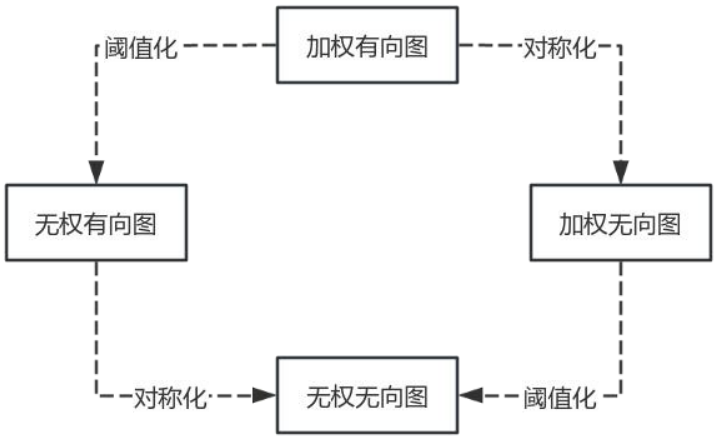


图 2.1 图的基本转化关系

加权有向图是最复杂的网络类型，即网络的边有方向性并且有权重。数学上表示为 $G=(V,E,W)$，V 是节点集，E 是有向边集，W 是权重函数 $W:E\rightarrow\mathbb{R}$。加权有向图能描述不对称的有强有弱的相互作用，例如金融市场，节点为金融组织机构，有向边为资金流向，权值为交易额。因此知道在链路预测中，需要同时考虑方向敏感特征（如入/出强度）和权重特征（如流量相似性）。

无权有向图只保留边上的方向信息，而不带有权值，表示为 $G=(V,E)$，其主要记录的是节点的纯粹连接信息与方向信息，代表节点之间的顺序信息或者逻辑关

系。比如说在一个任务调度图中，节点为各任务，边方向为任务执行先后顺序，无权有向图可以表达任务之间的依赖关系，而任务之间无需其他量化的信息。但是链路预测时需特殊处理方向性，例如在基于路径的特征中，只能沿边方向进行游走，一般将有向图转化为无向图后会损失方向的信息，因而这里推荐使用 HITS 算法等有向图。

加权无向图是计算连边权值的无向图，不指定方向，表示为 G=(V,E,W)，边权重通常是指边之间的联系度、代价等。社交网络的加权无向图，可以表示为用户之间的密切程度，连边权重可以根据用户之间的联系次数、用户共同爱好程度等。因边是无向的，即这种关系是无向的，意味着这种关系是相互的。

无权无向图是网络最简单的形式，代表两个节点之间的二元关系，表示为 *G=(V,E)*。虽然结构简单，但可以建模对称关系系统，比如合作网络和友谊网络。有研究表明，这类网络的巨连通分量通常包含 90%以上节点，链路预测中大多数经典方法（如 CN、AA 指标）都是为此类网络设计。当应用于其他网络类型时，需通过二值化处理，但是会损失权重或方向信息，这是此类方法的主缺点。

2.1.2 邻接矩阵

邻接矩阵（Adjacency Matrix）是图论中被用于表示网络拓扑结构的核心数学工具，其能够清晰地描述图中各个顶点之间的连接关系。

对于一个具有 n 个顶点的图$G=(V,E)$，这个图的邻接矩阵 A 是一个$n\times n$的矩阵。如果图 G 是无向图，当顶点v_i与v_j之间有边相连时，$A[i][j]=A[j][i]=1$；如果顶点v_i与v_j之间无边相连，则$A[i][j]=A[j][i]=0$。如果图 G 是有向图，当存在从顶点$\mathrm{v_i}$到$\mathrm{v_j}$的有向边时，$A[i][j]=1$；否则$A[i][j]=0$。对于加权图，矩阵元素$A[i][j]$的值可以是边(v_i,v_j)的权重；若不存在边(v_i,v_j)，则$A[i][j]$可设为一个特殊值，如无穷大∞。

假设有一个无向图 G，包含顶点 $V=\{v_1,v_2,v_3,v_4\}$ 和边 $E=\{(v_1,v_2),(v_1,v_3),(v_2,v_3),(v_3,v_4)\}$，其邻接矩阵为：

$$\begin{bmatrix}0&1&1&0\\1&0&1&0\\1&1&0&1\\0&0&1&0\end{bmatrix}$$

利用邻接矩阵可快速检测图中任意两个顶点的邻接关系，进而了解图的结构信息。在无向图中，如果两个顶点相邻，那么邻接矩阵中对应的元素 A_{ij} 和 A_{ji} 表现出对称性，所以在处理无向图时，只需存储邻接矩阵的上三角部分即可，这样

节省了存储空间，提高了计算效率。对于大型复杂网络，由于其连接稀疏，大部分顶点之间没有直接连接，邻接矩阵中大部分元素为零，这种矩阵被称为稀疏矩阵，其存储和计算方式需要特殊优化，方便充分利用其稀疏特性，提升处理大规模复杂网络的效率。

2.2 网络基本拓扑性质

依据网络的方向性与权重特性，网络可被归类为多种类型，而各类网络在计算拓扑属性时，所采用的方法也各不相同。鉴于本文研究聚焦于无权无向网络，本节所阐述的基本拓扑性质计算均以无权无向网络为背景展开。

2.2.1 网络大小与密度

网络规模通常是指一个网络中的节点(计算机、用户等)个数以及网络中的连线(连接关系)条数，是对一个网络大小的数值化描述，节点和边的数量越多，网络规模越大，网络的覆盖面与信息处理规模也就可能越大。网络密度是指一个网络中节点之间连接的密集程度，通用实际边数与可能边数的比值表示，对于无权无向网络，其计算公式为

$$D = \frac{m}{\frac{n(n-1)}{2}} \tag{2.1}$$

其中 n 为节点数量，m 为边的数量。D 的取值范围为 $0 \leq D \leq 1$，当 D=1 时，该网络为全联通网络，D=0 时网络中无边相连。网络密度越高，节点间的联系越紧密，信息越容易传播，在链路预测中，可能为算法提供更多关于节点关系的线索；而网络密度较低时，网络较为稀疏，链路预测可能越难以进行，需要更好的算法来预测节点的连接。

2.2.2 度与度分布

在无权无向网络中，节点的度（Degree）k_i 表示与该节点直接相连的边数，是一个描述节点连接能力的核心指标，若 A 为邻接矩阵，则其计算公式为

$$k_i = \sum_{j=1}^{N} A_{ij} \tag{2.2}$$

平均度是描述网络整体特征的一个重要指标。它是指网络中所有节点的度的平均值，计算公式为：

$$\text{平均度} = \frac{2m}{n} \tag{2.3}$$

其中 m 是网络中边的数量，n 是节点的数量。由于每条边连接两个节点，那

么度的总和就是每条边数了两次，所以用边数的两倍除以节点数就得到了平均度。

平均度是指网络中节点平均连接度。平均度越高，说明网络中节点之间的连接越紧密，网络的整体连通性和稳定性也相对较好；平均度越低，网络则相对较为稀疏，节点之间的联系不够紧密，可能导致网络信息无法传播，网络的稳定性也可能较差。例如，在好友圈中，平均度越高，说明网络中大多数人都能直接联系，网络中的信息可迅速传播；反之，平均度越大，说明网络中用户直接联系的少，信息可能要经过几个节点才能传播出去。

度分布 P(k) 描述网络中随机选取一个节点其度数为 k 的概率，可揭示网络的整体连接模式：随机网络的度分布服从泊松分布 $P(k) \approx e^{-\langle k\rangle}\langle k\rangle^k/k!$，而无标度网络则呈现幂律分布 $P(k) \sim k^{-\gamma}$（通常 $2 < \gamma < 3$），如 Internet 路由器网络 $\gamma \approx 2.3$。链路预测中，度分布特征直接影响相似性指标设计，对于幂律分布网络，需引入度归一化因子（如 Adamic-Adar 指标中的 $1/log\ k_z$）以避免高度节点的支配效应，同时度相关性分析可指导负采样策略的优化。

2.2.3 网络的平均路径长度与直径

在无权无向网络中，平均路径长度（L）定义为所有连通节点对之间最短路径长度的平均值，计算公式为

$$L = \frac{1}{N(N-1)}\sum_{i\neq j} d_{ij} \tag{2.4}$$

其中d_{ij} 表示节点 i 与 j 之间的最短路径长度（边数），$N = |V|$ 表示网络节点总数。网络直径（D）则是所有节点对间最长最短路径的长度，

$$D = \max_{i,j} d_{ij} \tag{2.5}$$

大多数真实网络具有显著的小世界特性，其平均路径长度远小于网络规模（通常$L \sim O(ln\ N) \ll N$），例如 Facebook 社交网络的 L≈4.7。这些指标在链路预测中描述的是一种网络的全局连接效率，当 L 较小时用局部结构特征（如共同邻居）就够了；而当 D 较大时就需要考虑全局路径特征。对于不连通网络，需用调和平均路径长度（L′）来避免无限值问题，这对预测潜在跨社区连接是很重要的。

$$L' = \frac{N(N-1)}{\sum_{i\neq j}\frac{1}{d_{ij}}} \tag{2.6}$$

2.2.4 聚类系数

在个体社交网络时，通常有一个规律，一个人的两个朋友之间比较容易联系到一起。因此，利用社会网络的聚集系数即可判断朋友联系的几率。聚类系数[20]

是衡量节点邻居之间联系紧密程度的重要指标。

假设节点 i 的度为 k_i，也就是说它有 k_i 个直接相连的邻居节点。假如这些邻居节点之间也两两相连，那么在理想情况下，这 k_i 个节点之间将存在 $k_i(k_i-1)/2$ 条边，但是在实际网络中，这种完全连接的情况较为少见。因此对于给定网络中度为 k_i 的节点 i，其聚类系数 C_i 的定义为

$$C_i = \frac{2E_i}{k_i(k_i-1)} \tag{2.7}$$

其中，E_i 表示节点 i 的 k_i 个邻居节点之间实际存在的边数。对于只有一个邻居节点或没有邻居节点的节点，显然 $E_i = 0$，从而 $C_i = 0$。根据公式 (2.7)，聚类系数 C_i 的取值范围为 $0 \le C_i \le 1$。当 $C_i = 0$ 时，表明节点 i 的任意两个邻居节点之间均无直接联系，或者节点 i 最多只有一个邻居节点；而当 $C_i = 1$ 时，则意味着该网络是全连接的。

2.2.5 同配系数

在无权无向网络中，同配系数[21]（Assortativity Coefficient）r 用于度量网络中度相关性，其定义基于 Pearson 相关系数如公式 2.8 所示：

$$r = \frac{M^{-1}\sum_{(i,j)\in E} k_i k_j - [M^{-1}\sum_{(i,j)\in E} \frac{1}{2}(k_i+k_j)]^2}{M^{-1}\sum_{(i,j)\in E} \frac{1}{2}(k_i^2+k_j^2) - [r\sum_{(i,j)\in E} \frac{1}{2}(k_i+k_j)]^2} \tag{2.8}$$

M 表示总边数，k_i和k_j分别表示边(i,j)两端节点的度数。当$r>0$时，网络呈现同配性，即表现为高节点度有较大概率彼此连边；当$r<0$时，则表现为异配性，就是高度节点有较大概率连接低度节点，出现在技术或生物网络。网络的链路预测效果同配性影响较大，在同配网络中，基于节点度的相似性指标（如 Degree Product）通常表现更好。在特征工程中，可通过引入度相关性特征（如$|k_i- k_j|$）来提升模型对网(i,j)的捕捉能力。对于大规模网络，可采用随机采样边集的方法来高效估算同配系数。

2.3 链路预测基础

2.3.1 链路预测问题描述

链路预测是复杂网络研究领域的热点之一，指的是根据网络已有的拓扑结构、节点属性信息以及历史信息，对网络中现在不存在但未来可能出现的连边，或当前因数据缺失而隐藏的真实连边进行预测。例如，在学术合作网络中，学者之间没有合作发表论文，他们之间没有连边，链路预测就是预测这些未合作学者未来

产生合作关系的可能性。

从数学建模角度来讲，无权无向网络可表示为 $G=(V,E)$，其中 V 是节点集合，$|V|=n$；E 是边的集合，$|E|=m$。对于任意两个节点 $u,v\in V$，若 $(u,v)\in E$，则表示节点 u 和 v 之间存在连边；若 $(u,v)\notin E$，则表示两者无直接连接。链路预测的目标是构建一个预测函数 $f:V\times V\rightarrow[0,1]$，该函数基于节点在网络中的拓扑位置、邻居节点等信息，输出节点 u 和 v 之间产生连边的概率值。当 $f(u,v)$ 越接近 1 时，表明两节点建立连接的可能性越高；反之，越接近 0 则可能性越低。

2.3.2 数据集的划分

再数据层面，对于无权无向网络链路预测问题，通常对网络中的节点对划分为正样本对与负样本对，其中正样本为已经存在连边的边对，负样本对从不存在连边的边对中随机抽取。因为存在连边的结点对数量远远小于不存在连边对的数量，为了避免数据不平衡往往采用随机抽样、欠采样、过采样等手段平衡样本量。例如，通过随机选取一定数量的未连边节点对作为负样本，与所有已连边节点对一起作为训练数据集，用于机器学习模型的训练。 假设，已知的连边集合为 E，随机地划分数据集为训练集为 E^T和测试集E^P，在这里有 $E^T\cup E^P=E$，$E^T\cap E^P=\emptyset$。

2.3.3 性能评价指标

在无权无向网络的链路预测任务中，性能评价指标用于量化评估模型预测结果的准确性与可靠性。根据链路预测任务特性，评价指标可分为基础分类指标和排序评价指标，从不同维度反映模型性能。

1. 基础分类指标

在链路预测中，模型输出通常为节点对是否存在连边的二元判断（是 / 否）或概率预测，基础分类指标基于预测结果与真实标签的对比，衡量模型分类性能。

（1）准确率（Accuracy）

准确率表示模型正确预测的样本数占总样本数的比例，计算公式为：

$$Accuracy=\frac{TP+TN}{TP+TN+FP+FN} \tag{2.9}$$

TP（True Positive）代表正确预测为正样本的数量（即实际存在连边且模型预测为存在连边的节点对数量）；TN（True Negative）代表正确预测为负样本的数

量（实际不存在连边且模型预测为不存在连边的节点对数量）；FP（False Positive）代表错误预测为正样本的数量（实际不存在连边但模型预测为存在连边的节点对数量）；FN（False Negative）代表错误预测为负样本的数量（实际存在连边但模型预测为不存在连边的节点对数量）。

准确率反映了模型整体预测的正确性，但在数据不平衡的情况下（如链路预测中负样本远远多于正样本），模型对少数类（潜在连边）的预测能力可能会被多数类样本主导结果而掩盖。

（2）精确率（Precision）

精确率衡量模型预测为正样本的结果中，真正为正样本的比例，公式为：

$$Precision = \frac{TP}{TP+FP} \tag{2.10}$$

精确率聚焦于模型预测的“精准度”，在链路预测中可用于评估模型预测的潜在连边中有多少是真实存在的。例如，在推荐系统的链路预测场景中，高精确率意味着推荐的潜在好友关系更可靠。

（3）召回率（Recall）

召回率也称查全率，计算模型正确预测的正样本数占实际正样本总数的比例，公式为：

$$Recall = \frac{TP}{TP+FN} \tag{2.11}$$

召回率反映模型捕捉到真实正样本的能力，在链路预测中用于衡量模型对所有潜在连边的发现程度。例如，在生物网络的链路预测中，高召回率有助于尽可能多地发现潜在的蛋白质相互作用关系。

（4）F1 分数

F1 分数是精确率和召回率的调和平均数，综合考虑两者表现，计算公式为：

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{2.12}$$

F1 分数取值范围为 0 到 1，数值越大模型越好，有一方是特别高或者特别低，F1 分值就非常低了，因此 F1 分数可作为平衡精确率与召回率的综合评价指标，适用于对模型综合性能要求较高的链路预测场景。

2. 排序性能指标

由于链路预测常需对所有节点对的连接可能性进行排序，以筛选高概率连边，排序评价指标用于评估模型输出的排序质量。

（1）AUC（Area Under the Curve of ROC）

AUC 就是受试者工作特征曲线（ROC 曲线）下的面积，ROC 曲线中假正率（$\mathrm{FPR} = \mathrm{FP}/\mathrm{FP}+\mathrm{TN}$）被作为横轴，真正率（$\mathrm{TPR} = \mathrm{TP}/\mathrm{TP}+\mathrm{FN}$）被作为纵轴，通过改变分类阈值绘制曲线。AUC 值越大，表示模型区分正样本和负样本的能力越强。AUC 取值范围为 0.5 到 1，0.5 表示模型预测结果等同于随机猜测，1 表示模型能完美区分正负样本。在链路预测中，AUC 常用于比较不同模型对节点对连接可能性的排序优劣。

（2）Precision@K

Precision@K 表示在模型预测的前 K 个节点对中，实际为正样本的比例，计算公式为：

$$Precision@K = \frac{\text{前 K 个预测中正确的边数}}{K} \tag{2.13}$$

这个指标关注模型在输出的 Top-K 预测结果中的精确性。在链路预测中，如果希望优先关注可能性最高的 K 条潜在连边，Precision@K 可用来评估这些高优先级预测结果的可靠性，帮助用户快速筛选出有价值的预测连边。

2.4 传统链路预测方法

2.4.1 基于相似性的方法

基于相似性的链路预测方法是传统链路预测中最常用的策略之一，其核心思想是通过量化节点之间的相似程度，来评估节点对之间存在连接的可能性。这类方法认为，在网络中具有更高相似性的节点之间更有可能形成边。

（1）共同邻居（Common Neighbors, CN）

简单直接地计算两个节点共同邻居的数量，如果节点 u 和节点 v 的共同邻居集合为 $N(u) \cap N(v)$，那它们的共同邻居数 $CN(u,v) = |N(u) \cap N(v)|$，共同邻居数越多代表两个节点在网络中的“社交圈子”重叠越多，意味着两个节点之间越容易在网络中产生联系 。就像社交网络中的两个人有共同的好多好友，也容易成为好友一样。

（2）Jaccard 系数

Jaccard 系数在共同邻居的基础上，考虑了节点邻居集合的并集大小，对共同邻居数进行了归一化处理，公式为

$$JC(u,v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|} \tag{2.14}$$

和 CN 相比 Jaccard 系数能更准确地描述了节点间相似性的相对程度，避免了

节点度的差异性的影响，比如说两个节点即使共同邻居数相同，但如果其中一个节点本身邻居数量很少，另一个节点邻居数量很多，Jaccard 系数会认为前者的相似性更高。

（3）Adamic-Adar 指数（AA）

该指数认为，若两个节点的共同邻居节点度数较小，那么这些共同邻居对于这两个节点建立连接的贡献更大。其计算公式为

$$AA(u,v)=\sum_{w\in N(u)\cap N(v)}\frac{1}{log\,(d(w))} \tag{2.15}$$

其中 d(w) 表示节点 w 的度数。在实际网络中，具有较少连接的共同邻居往往能更精准地反映节点间的相似性，Adamic - Adar 指数正是基于这一原理，对度数低的共同邻居赋予更高的权重。

（4）优先连接[22]（Preferential Attachment, PA）

优先连接指标基于“富者更富”的原则，认为节点的连接能力与其已有的连接数（度数）成正比。在网络演化过程中，新节点更倾向于连接到度数高的节点，从而形成“马太效应”。对于节点对(u,v)，优先连接指标定义为：

$$PA(u,v)=k_u\times k_v \tag{2.16}$$

其中k_u和k_v分别表示节点 u 和 v 的度数。该公式直观地反映了两个节点的“吸引力”乘积，度数越高的节点对越容易形成连接。

2.4.2 基于路径的方法

基于路径的链路预测方法从网络中节点间的路径信息出发，挖掘节点之间潜在的连接关系，考虑了节点间的间接联系，能够捕捉到网络结构中更复杂的依赖关系 。

（1）最短路径（Shortest Path）

最短路径法通过计算两个节点之间的最短路径长度来表示两个节点间的连接可能性，一般来说，最短路径长度越短则两个节点间的联系强度越高，将来产生连通的概率就越大。在实际，如果节点 u 和 v 之间的最短路径长度为 d_{uv}，可以根据不同的需求设定阈值，当 d_{uv} 小于一定值时，表示两节点可能形成连接。比如在交通网络中，两个地点之间的最短路径越短，也就是说它们之间建立新交通线路的可能性就越大。

（2）Katz 指标

Katz 指标综合考虑了节点间所有长度的路径信息，通过为不同长度的路径赋予不同的权重，来计算节点间的相似性。其计算公式为

$$s_{uv} = \sum_{l=1}^{\infty} \alpha^l \cdot |P_l(u,v)| \tag{2.17}$$

其中 α 是一个控制路径权重衰减的参数，$P_l(u,v)$ 表示节点 u 和 v 之间长度为 l 的路径集合。Katz 指标能够从全局角度捕捉节点间的复杂关系，在处理具有多种连接模式的网络时表现较好。

2.5 基于机器学习的链路预测算法

机器学习作为诸多领域中的交叉学科，在链路预测领域发挥着重要作用，它能够从复杂的网络数据中自动学习模式，进而提升链路预测的准确性和可靠性。主要介绍与本文关系紧密的的逻辑回归（Logistic Regression，LR）算法和 LightGBM 算法。

2.5.1 逻辑回归算法

逻辑回归（Logistic Regression）是一种经典的广义线性回归模型，虽然叫为“回归”，但实一般用于二分类问题，在链路预测中可以判断节点对之间是否会形成边 。逻辑回归基于逻辑函数（sigmoid function）进行建模，公式为：

$$\sigma(z) = \frac{1}{1+e^{-z}} \tag{2.18}$$

其中，$z = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n$，$\mathrm{w_i}$是模型的参数，$x_i$是输入特征。在链路预测中，这些特征可以是节点对的相似性指标，比如共同邻居数量、Jaccard 系数等。

逻辑回归通过最大似然估计（Maximum Likelihood Estimation）来确定参数w_i的值，目标是最大化观测数据出现的概率，逻辑回归的优点优点是计算简单，模型的可解释性强。通过查看参数w_i的大小和正负，可以很直观地判断每个特征对于预测值的影响方向和大小，正的w_i表示对应的特征与形成链路的可能性正相关。缺点是对数据中的非线性关系捕捉能力较弱，网络结构越复杂越不能将数据中的信息全部挖掘出来。

2.5.2 LightGBM 算法

LightGBM 是由微软开发的梯度提升框架，基于决策树算法，在处理大规模数据和复杂模型训练时能够很好地发挥其优势。LightGBM 通过使用像直方图算法

（Histogram Algorithm）、单边梯度采样（GOSS）和互斥特征捆绑（EFB）等这些，使得模型训练速度更快，训练质量更高。

直方图算法会把连续的特征值离散化为有限个区间，用直方图的形式存储数据，降低内存占用，加速计算。单边梯度采样根据梯度的大小对数据进行采样，保留梯度较大的数据，舍弃部分梯度较小的数据，在不损失太多精度的情况下，节省计算量。互斥特征捆绑将互斥的稀疏特征绑在一起，降低特征维度，提升训练速度。

在链路预测中，LightGBM 能够自动地学习数据中的复杂非线性关系，通过构建多棵决策树并逐步拟合残差，提升了模型的预测能力。和传统决策树算法比起来，能更有效地处理大规模网络数据，避免了过拟合问题，在训练速度上也有一定的提升。但由于是树的结构，模型的可解释性相对逻辑回归来说比较弱。

第 3 章 算法原理与代码实现

3.1 算法分析与选择

在机器学习算法的选择中，LR（Logistic Regression）算法算法是经典之选。LR 作为线性分类器的代表，模型的优势是简单直观，核心是通过线性组合特征，并利用逻辑函数映射输出概率，计算过程高效，处理大规模数据时也有很好的表现，同时清晰的参数含义让模型决策过程易于理解逻辑回归算法。

LLSLP 算法是将 LR 算法与 LightGBM 算法相结合，一方面 LightGBM 强大的非线性建模能力能够有效捕捉数据中复杂的非线性关系，另一方面通过模型集成的方式，进一步提升算法的性能。

二者进行对比可以看出 LLSLP 算法的性能优势。

3.2 逻辑回归算法

3.2.1 算法设计思想

逻辑回归（Logistic Regression，LR）是二分类问题常用的统计学方法。在链路预测中，逻辑回归算法的核心思想是，将节点对的特征与它们之间是否存在边的概率联系起来，通过学习特征与目标之间的关系来预测未知链路的存在性。

在链路预测问题中，逻辑回归将输入特征设置为节点对的特征（如共同邻居数、Jaccard 系数、Adamic-Adar 指数等），将节点对之间是否存在边作为目标变量（存在边为正样本，不存在边为负样本）。通过最大化似然函数来获取模型参数，使模型能够根据输入特征对节点对之间是否存在边这个概率进行预测，最终将概率根据阈值映射为类别标签来进行链路预测。

具体算法实现如下：

算法 1：LR 链路预测算法

```
函数 train_logistic_regression(X_train, y_train, X_test, y_test):
    # 特征标准化
    初始化标准化器 scaler
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    # 模型训练
    初始化逻辑回归模型 model
```

```
    model.fit(X_train_scaled, y_train)
    # 预测
    y_pred_proba = model.predict_proba(X_test_scaled)[:, 1]
    y_pred = model.predict(X_test_scaled)
    # 评估
    AUC = roc_auc_score(y_test, y_pred_proba)
    Accuracy = accuracy_score(y_test, y_pred)
    Precision = precision_score(y_test, y_pred)
    Recall = recall_score(y_test, y_pred)
    F1 = f1_score(y_test, y_pred)
    # 返回结果
    返回 {
        'model': model,
        'scaler': scaler,
        'auc': AUC,
        'accuracy': Accuracy,
        'precision': Precision,
        'recall': Recall,
        'f1': F1,
        'y_pred_proba': y_pred_proba,
        'y_pred': y_pred
    }
```

3.2.2 代码实现

1. 特征标准化

使用 StandardScaler 对特征进行标准化处理，使每个特征的均值为 0，标准差为 1，有助于提高逻辑回归模型的训练效率和性能；fit_transform 计算训练集的均值和标准差，用得到的均值、标准差标准化训练集特征；transform 使用训练集的均值和标准差对测试集特征进行标准化。具体代码如下：

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

2. 模型参数设置

初始化逻辑回归模型 LogisticRegression，参数设置如表 3.1 所示。使用标准化后的训练数据和标签训练模型

表 3.1 逻辑回归算法的参数配置

参数设置	参数解释
random_state=SEED	固定随机种子，确保结果可复现
max_iter=1000	设置最大迭代次数为 1000，以确保模型能够收敛
class_weight='balanced'	自动调整权重以处理不平衡数据
C=1.0	设置正则化强度为 0.1，以防止过拟合
solver='liblinear'	选择适合小数据集的求解器

3. 预测

使用训练好的模型对测试集进行预测，得到预测概率 y_pred_proba 和预测类别 y_pred。predict_proba 方法返回每个样本属于正类的概率。predict 方法直接返回预测的类别标签。具体代码如下：

```
y_pred_proba = model.predict_proba(X_test_scaled)[:, 1]#返回正样本的概率
y_pred = model.predict(X_test_scaled)#模型的预测方法，
```

4. 评估

计算 AUC 值，评估模型对正类的预测能力；计算准确率，评估模型的整体分类性能；计算精确率，评估模型预测为正类的样本中实际为正类的比例；计算召回率，评估模型实际为正类的样本中预测为正类的比例；计算 F1 分数，综合评估模型的精确率和召回率。具体代码如下：

```
auc = roc_auc_score(y_test, y_pred_proba)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

5. 返回结果

将训练好的模型、特征标准化器以及评估指标封装成字典返回，便于后续的模型保存和性能对比。具体代码如下：

```
results = {
```

```
        'model': model,
        'scaler': scaler,
        'auc': auc,
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'y_pred_proba': y_pred_proba,
        'y_pred': y_pred
}
return results
```

3.3 LLSLP 算法

3.3.1 算法设计思想

本文所实现的 LLSLP 是一种高效的链路预测方法，把社交网络中的链路预测问题视为一个二分类问题进行预测。LLSLP 是一种基于堆叠（Stacking）集成学习框架的链路预测方法，通过逻辑回归（Logistic Regression）和两个 LightGBM 模型的优势互补，提升预测性能。

LLSLP 采用多模型协同架构，逻辑回归和两个不同参数的 LightGBM 模型组成第一层分别学习线性关系和非线性特征，第二层通过 LightGBM 元模型整合第一层模型的输出概率，进一步优化预测结果。利用堆叠框架自动学习不同模型的特征表示，避免了手工设计复杂融合策略。

具体算法实现如下：

算法 2：LLSLP 链路预测算法

输入: 网络 G=(V,E), 节点对集合 P, 特征集 F, 交叉验证折数 K

输出: 节点对 P 的链路存在概率向量 Y

```
函数 LLSLP(G, P, F, K=5):
    // 第一阶段: 特征工程
    X ← ExtractMultiDimensionalFeatures(G, P, F)   // 提取多维度特征
    X_scaled ← FeatureNormalization(X)   // 特征标准化处理
    // 第二阶段: 交叉验证训练与元特征生成
```

```
meta_features ← zeros(|P|, 2)   // 初始化元特征矩阵
base_models ← []   // 存储各折的基模型
for k ← 1 to K do:
    (X_train, y_train), (X_val, y_val) ← KFoldSplit(X_scaled, y, k)
    // 训练基模型
    lr_model ← TrainLogisticRegression(X_train, y_train)   // LR 模型
    lgb_model ← TrainLightGBM(X_train, y_train)   // LGB 模型
    base_models.append((lr_model, lgb_model))
    // 生成元特征
    meta_features[val_indices, 0] ← lr_model.predict_proba(X_val)[:, 1]
    meta_features[val_indices, 1] ← lgb_model.predict(X_val)
end for
// 第三阶段: 元模型训练
meta_model ← TrainMetaModel(meta_features, y)   // 训练元分类器
// 第四阶段: 测试阶段
final_lr_model ← TrainLogisticRegression(X_scaled, y)   // 最终 LR 模型
final_lgb_model ← TrainLightGBM(X_scaled, y)   // 最终 LGB 模型
// 生成测试元特征
test_meta_features ← [
    final_lr_model.predict_proba(X_test_scaled)[:, 1],
    final_lgb_model.predict(X_test_scaled)
]
// 最终预测
y_pred_proba ← meta_model.predict_proba(test_meta_features)
return y_pred_proba
```

3.3.2 Stacking 集成学习模型架构

1. 模型设计思想

Stacking 集成学习的实质就在于通过分层建模实现多模型优势互补，设计思想来自“模型偏差 - 方差分解”理论：不同类型的基础模型一般在捕捉数据特征

时存在差异，通过组合这些模型，可以组合模型以消减一个模型的方差或偏差来提升组合模型的泛化能力。在链路预测中，网络结构特征既包含线性关系，比如度数乘积、共同邻居数的线性组合，也包含非线性关系，比如聚类系数与最短路径的交互作用，一个模型很难同时解释二者，因此 LLSLP 模型采用双层架构。

第一层是基模型层，选具有不同归纳偏置的基础学习器，分别学习特征空间的不同维度。这边选取两个模型：线性模型（逻辑回归）和非线性模型（LightGBM）。线性模型假设特征与标签之间存在线性关系，通过权重系数直接建模特征的线性组合效应，可以学习如“优先连接原理（PA 特征 = 度数乘积）”等线性关联较强的模式。非线性模型基于树结构的集成模型，学习特征间的高阶交互作用（如“高聚类系数节点与短路径节点的连接倾向”），使用梯度提升机制拟合复杂非线性关系。通过这种组合，基础模型层可从线性与非线性两个视角对网络特征进行编码，生成互补的预测结果。

第二层是元模型层，元模型的目标是学习基础模型的预测偏差，通过“二次建模”实现对复杂模式的精细化捕捉，就是将基础模型对样本的概率预测结果，构成元特征空间作为输入。输出就是融合后的最终预测概率，本质是对基础模型“预测误差”的建模 。例如，当逻辑回归与 LightGBM 对某样本的预测概率差异较大时，元模型可通过学习该差异判断是否需要修正预测结果。并且采用 5 折交叉验证生成元特征，即基模型在交叉验证过程中对验证集的预测结果，以保证基模型的训练集过拟合不会导致元模型产生偏差，，确保元特征的泛化性。

具体实现架构图 3.1 所示。

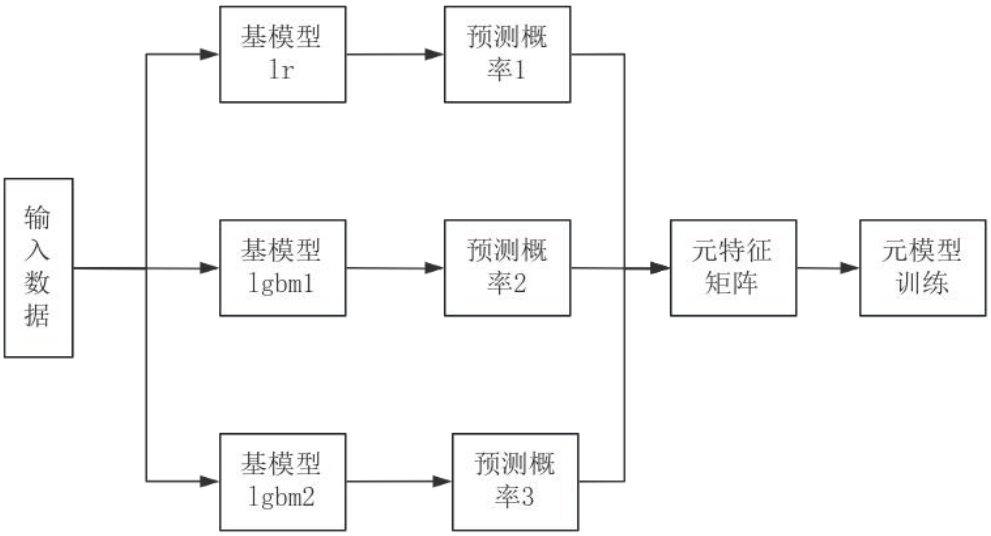


图 3.1 LLSLP 方法模型实现架构

2. 模型组件与参数配置

（1）基础模型

基础模型的选择需满足“多样性”与“性能均衡”原则，确保其预测结果具有互补性而非冗余性。LLSLP 采用以下组合：

①逻辑回归（LR）

作为线性基线模型，提供特征线性组合的基准预测，同时可通过系数显著性分析评估单一特征的重要性（如共同邻居数的权重大小）。LR 算法的参数配置如表 3.2 所示。

表 3.2 LR 算法的参数配置

参数设置	参数解释
random_state=SEED	固定随机种子，确保结果可复现
max_iter=1000	设置最大迭代次数，防止优化过程因迭代不足而提前终止
penalty='l2'	使用 L2 正则化（岭回归），防止过拟合
C=1.0	正则化强度的倒数，较小的值表示更强的正则化。默认值 1.0 平衡了模型复杂度与泛化能力。
solver='lbfgs'	拟牛顿法求解器，适用于多分类和大规模数据集，支持 L2 正则化

②LightGBM1

较浅的树（max_depth=5）和适中的学习率，快速捕捉数据的主要模式。LightGBM1 算法的参数配置如表 3.3 所示。

表 3.3 LightGBM1 算法的参数配置

参数设置	参数解释
n_estimators=100	树的数量，控制模型复杂度。较小的值（100）防止过拟合，适合基础模型。
learning_rate=0.1	较大的学习率，加速收敛但可能降低精度
max_depth=5	树的最大深度，限制模型复杂度，防止过拟合
num_leaves=31	每棵树的最大叶子节点数，与 max_depth 协同控制树的复杂度
min_child_samples=20	叶子节点所需的最小样本数，避免过拟合
subsample=0.8	训练每棵树时随机采样 80% 的数据，增加随机性和泛化能力。
colsample_bytree=0.8	每棵树随机采样 80% 的特征，减少特征间的依赖
**gpu_params：	若 GPU 可用，则启用 GPU 加速

③LightGBM2

更深的树（max_depth=7）和更小的学习率，挖掘更复杂的非线性关系。LightGBM2 算法的参数配置如表 3.4 所示。

表 3.4 LightGBM2 算法的参数配置

参数设置	参数解释
n_estimators=200	增加树的数量，提升模型容量
learning_rate=0.05	较小的学习率，配合更多树，提高模型稳定性和精度
max_depth=7	树的最大深度，限制模型复杂度，防止过拟合
num_leaves=63	更多的叶子节点，增强模型表达能力
min_child_samples=10	较小的叶子节点样本数，允许更细粒度的分裂
subsample=0.6	更激进的数据采样，增加随机性，降低过拟合
colsample_bytree=0.6	更激进的特征采样，增强模型鲁棒性
reg_alpha=0.1 和 reg_lambda=0.1	L1 和 L2 正则化，进一步防止过拟合

（2）元模型

采用 LightGBM 算法，支持直接输入概率型特征，不需要额外的归一化，GPU 加速可处理高维元特征，基础模型为 3 个时，元特征维度为 3，可以通过决策树分裂可学习基础模型预测结果间的非线性组合，“当 LR 预测概率> 0.6 且 LGBM1 预测概率 ＜0.5 时，元模型判断为负样本”。

在训练阶段，用 5 折交叉验证生成元特征，训练集被划分为成了 5 个子集，每个基础模型在 4 个子集上训练，在剩余 的 1 个子集上生成预测概率，最终拼接所有子集的预测结果作为元模型的训练输入。这样可以避免基础模型“记忆”训练集，确保了元特征仅包含模型的泛化性预测信息。元模型的参数变量如表 3.5 所示。

表 3.5 元模型的参数配置

参数设置	参数解释
n_estimators=500	增加树的数量，因为元特征较稳定
learning_rate=0.05	较小的学习率，提高模型稳定性和精度
max_depth=6	中等深度限制树的复杂度，防止对基础模型的输出过度拟合
num_leaves=63	更多的叶子节点，增强模型表达能力
num_leaves=40	控制树的分支数量，与 max_depth 协同作用，避免生成过于复杂的树
min_child_samples=30	要求每个叶子节点至少包含 30 个样本，进一步约束树的生长，增强泛化能力
colsample_bytree=0.6	更激进的特征采样，增强模型鲁棒性
reg_alpha=0.1 和 reg_lambda=0.1	L1 和 L2 正则化，进一步防止过拟合
subsample=0.8	每次迭代随机采样 80% 的数据，增加训练的随机性，提升模型鲁棒性
colsample_bytree=0.8	每棵树随机采样 80% 的特征，减少特征间的依赖性，降低过拟合风险

3.3.3 数据生成与预处理

在进行链路预测之前，首先需要对网络数据进行预处理。本研究中，网络数据以图的形式表示，节点表示实体，边表示节点之间的连接关系。数据预处理的主要步骤包括：加载图数据，生成正负样本以及特征提取。

1. 加载图数据

链路预测任务中，网络数据以图数据的形式给出，节点表示节点实体，边表示边的关系。为了支持不同格式的图数据，设计了多个格式图的加载模块，能够加载大部分常用格式的图数据。

一种是文本格式，每行表示一条边，边的两个节点以空格分隔。例如 1 2 表示节点 1 和节点 2 之间存在一条边。读入 Facebook 的图数据时，文件一行为一个单位，先过滤掉#开头的注释行，将两个节点对加入图形中。

Matrix Market 格式是一种标准化的矩阵存储格式，一般用来描述稀疏矩阵。在图数据中，通过行和列的形式描述稀疏矩阵中每一条边的对应关系。读取 Matrix Market 格式的图数据时，会跳过以%开头的注释行，将剩下的每一行中节点的对，添加到图中。

load_graph 函数被用作加载图数据。根据扩展名自动的识别文件格式，并进行解析处理，如果无法根据扩展名进行识别，则寻找目录下的 out.*文件或者.mtx 文件，并对其加载图数据。

2. 生成正负样本与数据集划分

于训练、测试的样本集构造中需要生成和正样本相同的负样本进行平衡, 负样本的结点对不能是自环的, 即不能是相同的两个结点, 结点对不能是已存在的边。当网络规模较大时，像 Facebook 数据集有 4039 个节点，遍历所有的边计算的成本很高。代码中通过 random.randint 随机生成节点索引，利用 G.has_edge 快速判断边是否存在，结合 set 结构存储已采样非边，防止重复采样。负样本仅基于原始网络生成，不依赖训练 / 测试集划分结果，确保训练阶段不会学习到测试集真实边信息。避免信息泄露。以下是具体的代码实现：

```
def generate_dataset(G):
    """生成训练和测试数据集"""
    # 获取所有边（正样本）
    all_edges = list(G.edges())
    positive_examples = all_edges.copy()
```

```
# 生成负样本（不存在的边）
negative_examples = []
all_nodes = list(G.nodes())
n_nodes = len(all_nodes)
# 添加进度条
print("生成负样本...")
with tqdm(total=len(positive_examples), desc="负样本生成") as pbar:
    # 生成与正样本数量相当的负样本
    while len(negative_examples) < len(positive_examples):
        # 随机选择两个节点
        i = random.randint(0, n_nodes - 1)
        j = random.randint(0, n_nodes - 1)
        node_i, node_j = all_nodes[i], all_nodes[j]
        # 确保不选择自环和已存在的边
        if node_i != node_j and not G.has_edge(node_i, node_j):
            negative_examples.append((node_i, node_j))
            pbar.update(1)
            if len(negative_examples) >= len(positive_examples):
                break
```

为评估模型的泛化能力，需将正负样本都按照 8 : 2 的比例划分训练集与测试集，模拟 “利用历史网络推断未来连接” 的真实场景。

3. 特征提取与标准化

在 LLSLP 方法中，特征提取是关键步骤之一，其质量直接影响模型性能，通过提取节点对间的多维度结构特征来表征他们连接的可能性。

本方法实现的特征提取函数提取了 8 个相似性指标。基于共同邻居的结构相似性指标，包括 CN、Jaccard 系数 (JC) 和 Adamic-Adar 指数 (AA)，因为“共同邻居越多，两个点越有可能连接到一起”。PA 是基于节点度的优先连接指标，假设节点连接概率与度的乘积成正比，捕捉了网络中的 "富者更富" 现象。还包括基于节点属性的特征，节点度 (deg_i, deg_j) 是局部连接密度，聚类系数 (cc_i, cc_j) 衡量节点邻居的互连程度。基于路径的特征，Katz 中心性是计算所有截断后的路

径长度，考虑节点间边的存在；最短路径长度则反映节点间的距离。对无法达到的节点和无法连通的图都进行特殊处理以保证特征计算结果的可靠性。具体模型特征见表 3.6。

表 3.6 模型特征

特征名称	符号	物理意义
共同邻居数	cn	两节点共同邻居的数量
Jaccard 系数	jaccard	共同邻居数与邻居并集大小的比值
Adamic-Adar 指数	aa_index	对低度数共同邻居赋予更高权重的相似性指标
优先连接	pa	两节点度数的乘积，衡量连接倾向
节点度数	deg_i/deg_j	节点 i/j 的度数
聚类系数	cc_i/cc_j	节点 i/j 的局部聚类系数
Katz 指数	Katz_i	两节点间所有路径的加权和（短路径权重更高）
最短路径长度	shortest_path	两节点间最短路径长度（若不连通则为直径）

具体实现代码如下所示：

```
def extract_features(G, node_i, node_j):
    common_neighbors = list(nx.common_neighbors(G, node_i, node_j))
    cn = len(common_neighbors)
    union_size = len(set(G.neighbors(node_i)).union(set(G.neighbors(node_j))))
    jaccard = cn / union_size if union_size > 0 else 0
    # Adamic-Adar 指数
    aa_index = sum(1 / np.log(G.degree(n)) for n in common_neighbors if G.degree(n) > 1)
    pa = G.degree(node_i) * G.degree(node_j)
    deg_i, deg_j = G.degree(node_i), G.degree(node_j)
    try:
        cc_i = nx.clustering(G, node_i)
        cc_j = nx.clustering(G, node_j)
    except:
        cc_i, cc_j = 0, 0
    try:
```

```
        katz_i = sum(1 / (len(p) - 1) for p in nx.all_simple_paths(G, source=node_i, target=node_j, cutoff=3))
    except:
        katz_i = 0
    try:
        shortest_path = nx.shortest_path_length(G, source=node_i, target=node_j)
    except:
        shortest_path = nx.diameter(G) if nx.is_connected(G) else -1
    return [cn, jaccard, aa_index, pa, deg_i, deg_j, cc_i, cc_j, katz_i, shortest_path]
```

模型输入特征包含多种量纲差异显著的指标，比如度数可是数千， Jaccard 系数介于 0-1，如果直接输入模型容易造成优化偏差，需进行标准化处理来消除不同特征之间的量纲差异，提高模型的训练效果。

采用 sklearn 的 StandardScaler，其数学表达式如 3.1 所示：

$$x' = \frac{x-\mu_{train}}{\sigma_{train}} \tag{3.1}$$

其中 μ_{train} 为训练集特征的均值，σ_{train} 为训练集特征的标准差。通过用训练集的样本的特征的均值和训练集的样本的特征的标准差作为统计量的测度，对测试集的数据进行变换，测试集不参与统计是为了避免测试集中数据的泄漏造成评估偏差，具体实现代码如下所示：

```
print("标准化特征...")
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

3.3.4 模型预测与评估

1. 模型训练与预测

将标准化后的特征输入到 Stacking 模型中进行训练。在训练中基学习器首先对数据进行学习，生成中间特征，元学习器再基于这些中间特征去预测最终的输出。通过这种方式 LLSLP 算法能够充分利用各基学习器的优势，提高链路预测的准确性。

```
stacked_model.fit(X_train_scaled, y_train)
```

在训练过程中，基学习器会通过 5 折交叉验证进行基模型的训练和预测，将每个基模型在每一折上的预测概率进行合并，生成元特征矩阵，元学习器基于这些中间特征进行学习。训练完成后，使用训练好的模型对测试数据进行预测。在代码中，预测过程如下：

```
y_pred_proba = stacked_model.predict_proba(X_test_scaled)[:, 1]
y_pred = stacked_model.predict(X_test_scaled)
```

predict_proba 方法返回每个样本属于每个类别的预测概率。[:, 1]表示取正样本的预测概率。predict 方法返回每个样本的预测类别

2. 模型评估

为了评估模型的性能，使用了多种指标，包括 AUC 值、准确率、精确率、召回率和 F1 得分。在代码中，评估过程如下：

```
auc = roc_auc_score(y_test, y_pred_proba)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

roc_auc_score 计算 AUC 值，表示模型预测正样本的概率大于负样本的概率的比率。accuracy_score 计算准确率，表示模型正确预测的样本数占总样本数的比例。precision_score 计算精确率，表示模型预测为正样本的样本中实际为正样本的比例。recall_score 计算召回率，表示模型正确预测为正样本的样本数占实际正样本总数的比例。f1_score 计算 F1 得分，表示精确率和召回率的调和平均值，综合考虑了精确率和召回率的平衡。

第 4 章 实验结果与分析

4.1 实验环境

本文采用 Python 编程，利用 PyCharm 编译器进行编译。Python 版本为 3.9，PyTorch 和 CUDA 环境版本为 torch1.8.1+cu111。使用 NVIDIA GeForce RTX 3090 显卡。

4.2 数据集

本次实验中采用了 5 个数据集，分别是 Zachary_Karate 网络，Jazz 网络，Celegans 网络，Dolphins 网络和 Facebook 网络。表 4.1 是对数据集的介绍。

表 4.1 数据集的介绍

数据集名称	节点数	边数	描述
Zachary_Karate	34	78	Zachary 空手道俱乐部社交网络，记录成员间的交互。其中包含了 34 位会员，会员之间的频繁交往形成了这些连边。
Jazz	198	2742	爵士音乐家合作网络，节点为音乐家，由 198 个独立音乐家构成。
Celegans	452	2025	线虫神经网络，每个节点代表一个特定的神经元，边表示神经元之间的交互，当两个神经元进行交互时，他们之间就会形成一条相互联系的边。
Dolphins	62	159	新西兰的一个海豚群落，有 62 只海豚之间频繁联系，当他们一起被发现时，他们之间形成一个连边。
Facebook	4039	88234	Facebook 数据集是由 SNAP 对部分调查者使用 Facebook 应用程序收集到的数据集,包含用户之间的好友关系。

4.3 实验结果与分析

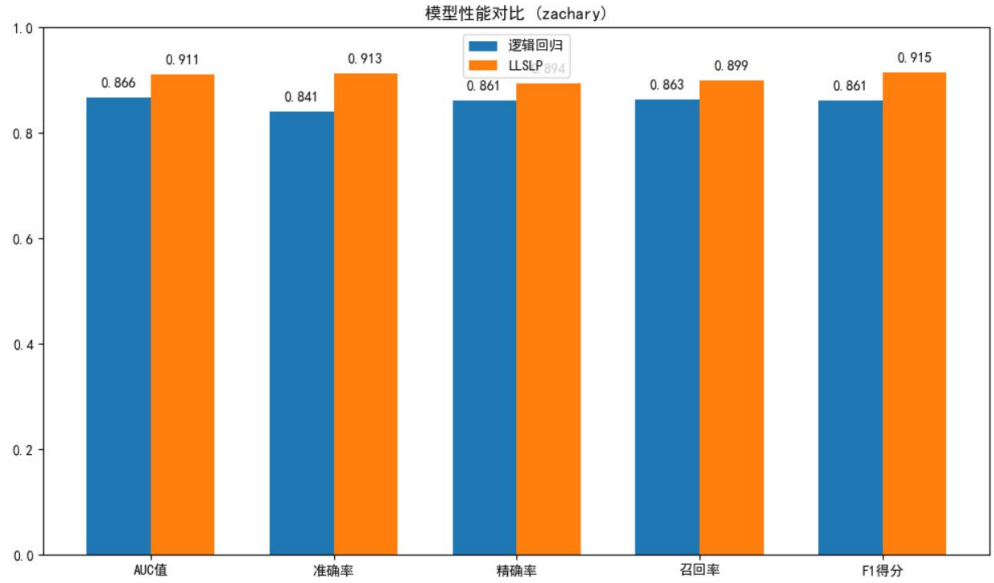
4.3.1 实验结果

采用了 5 个标准的评估指标来量化算法的预测性能：AUC、Accuracy、Precision、Recall、F1，实验记录了 5 个真实网络中 5 个不同指标的表现。图 4.1 展示了在 5 个数据集下 LR 算法和 LLSLP 方法在 5 项性能指标下的对比。

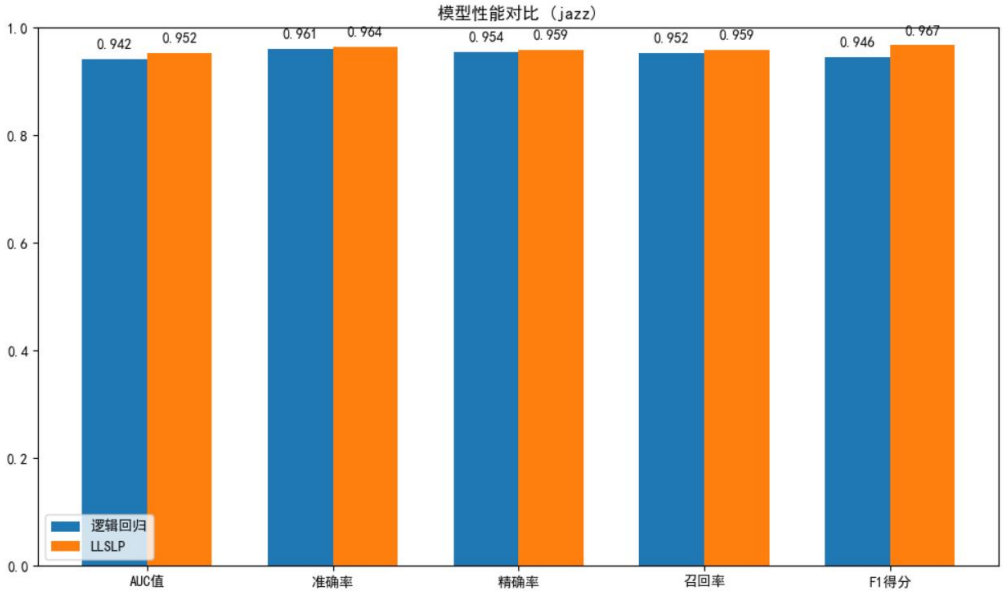
4.3.2 实验结果分析

表 4.2~4.6 给出了 AUC、Accuracy、Precision、Recall、F1 方面的对比分析，并分别给出了 LLSLP、LR 的对比分析。图中结果表明 LLSLP 相较与 LR 算法在各项性能上都有所提升，在一定程度上优于 LR 算法。

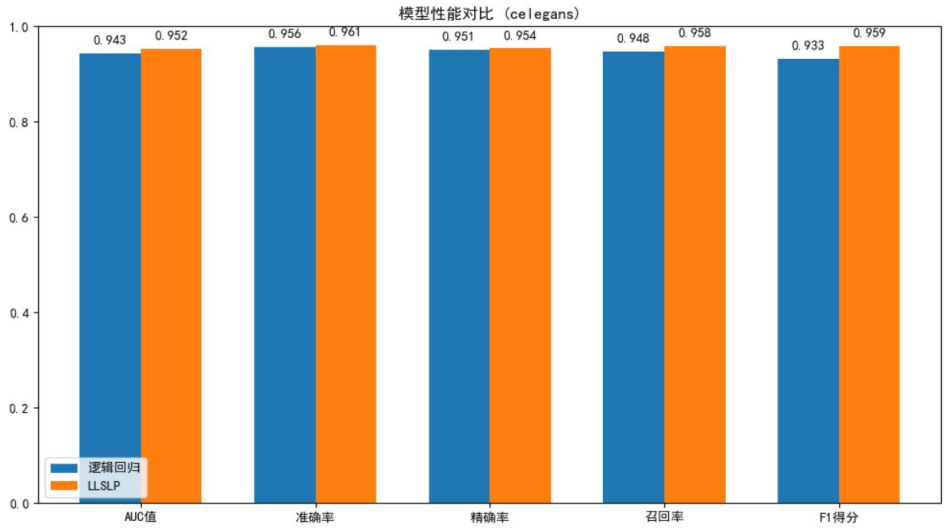
因为逻辑回归是一种线性分类模型，它假设特征与目标变量之间存在线性关系，对于复杂的非线性关系，逻辑回归可能无法很好地捕捉。LLSLP 是一个堆叠模型，结合了多个基础模型（如逻辑回归、LightGBM）和一个元模型（也是 LightGBM），可以捕捉到更复杂的特征关系和模式，通过集成多个基础模型，减少了单一模型的过拟合风险，提高了模型的泛化能力。



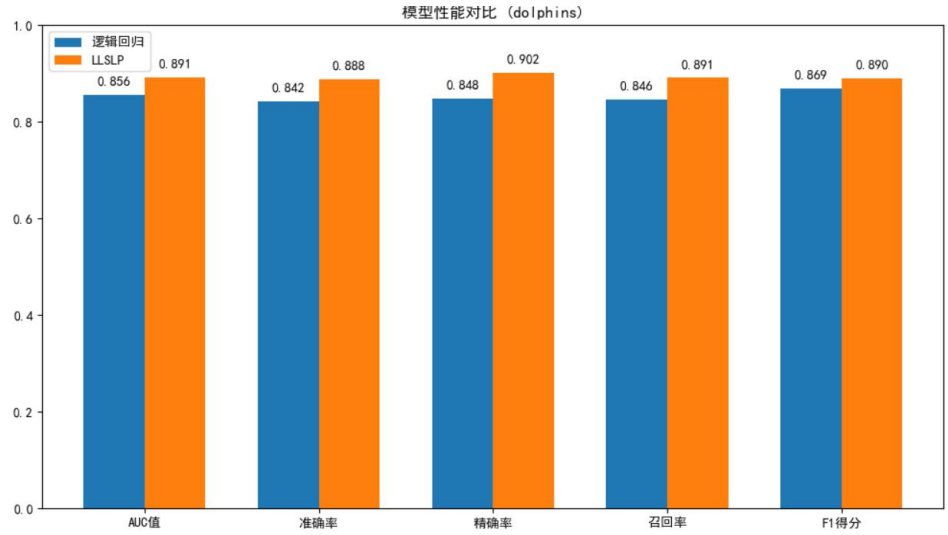
（a）Zachary



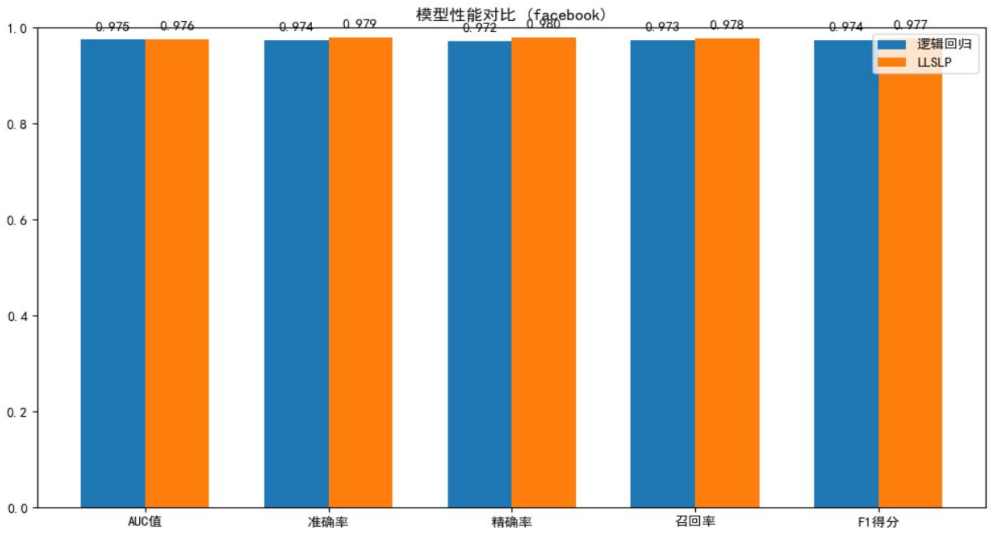
（b）Jazz



（c）celegans



（d）dolphins



（e）facebook

图 4.1 模型性能对比

表 4.2 五个网络中的 AUC 指标结果

算法	Zachary_Karate	Jazz	Celegans	Dolphins	Facebook
LR	0.8663	0.9421	0.9433	0.8564	0.9747
LLSLP	0.9114	0.9523	0.9521	0.8914	0.9764

表 4.3 五个网络中的 Accuracy 指标结果

算法	Zachary_Karate	Jazz	Celegans	Dolphins	Facebook
LR	0.8405	0.9607	0.9561	0.8423	0.9744
LLSLP	0.9127	0.9641	0.9609	0.8876	0.9764

表 4.4 五个网络中的 Precision 指标结果

算法	Zachary_Karate	Jazz	Celegans	Dolphins	Facebook
LR	0.8614	0.9541	0.9512	0.8479	0.9718
LLSLP	0.8943	0.9587	0.9537	0.9016	0.9802

表 4.5 五个网络中的 Recall 指标结果

算法	Zachary_Karate	Jazz	Celegans	Dolphins	Facebook
LR	0.8627	0.9521	0.9476	0.8456	0.9732
LLSLP	0.8994	0.9593	0.9577	0.8909	0.9776

表 4.6 五个网络中的 F1 指标结果

算法	Zachary_Karate	Jazz	Celegans	Dolphins	Facebook
LR	0.8607	0.9456	0.9327	0.8692	0.924
LLSLP	0.9149	0.9671	0.9589	0.8903	0.9772

第 5 章 演示软件的设计与实现

为了方便演示模型的效果，本文设计了一个图形界面，本章将详细介绍该系统的设计与实现。

5.1 开发平台

操作系统：Windows 11

5.1.1 前端开发环境

开发环境：Node.js 18.20.4，pnpm 10.10.0

编程语言：JavaScript

开发过程中使用的框架主要如表 5.1 所示：

表 5.1 开发使用框架清单

核心框架	构建工具	UI 组件库	HTTP 客户端
Vue 3	Vite 4	Element Plus	Axios

5.1.2 后端开发环境

后端采用 Flask 框架[23]搭建服务器。Flask 作为轻量级 Python Web 框架具有灵活简洁的特点适合快速开发中小型 Web 应用。后端代码集成多个 Python 库，其中 networkx 用于图数据处理分析，joblib 用于模型保存加载，torch 支持深度学习计算，lightgbm 用于梯度提升模型训练，这些库为链路预测模型的训练和预测提供有力支撑。

5.2 系统设计

5.2.1 架构设计

系统采用前后端分离的架构设计，前端使用 Vue 实现用户界面，后端使用 Flask 框架提供 API 服务。前后端通过 RESTful API 进行通信，实现数据的交互和业务逻辑的处理。架构具有良好的可扩展性和维护性，前端和后端可以独立开发、测试和部署，提高开发效率。

5.2.2 系统介绍

系统首页对系统主要支持算法进行展示，并且对本次实验所采用的数据集进行介绍。本系统主要完成两个功能，分别是链路预测和模型对比功能。

在链路预测时，用户首先需要选择模型，即数据集以及算法模型，然后选择需要预测的节点对，点击开始预测后在页面中展示预测结果。预测结果中将展示所需要预测节点对的连接概率和预测结果，并将所有预测结果进行分析展示。设置了导出结果按钮，可以将结果保存为 csv 文件，对结果进行查看。

在模型对比页面，需要选择数据集以及要比较的模型，可以对测试的节点数进行设置，点击开始比较按钮后，在界面的下方会对模型对比（包括性能指标对比，预测速度对比以及预测结果对比）结果进行直观的展示，同样，也可以对对比结果进行导出，保存为 csv 文件。

5.2.3 系统功能模块设计

基于机器学习方法的链路预测演示软件的主要功能如图 5.1 所示。

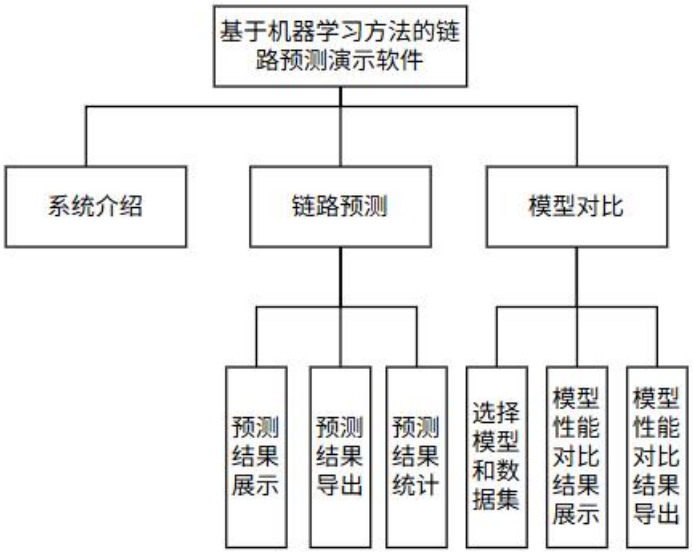


图 5.1 基于机器学习方法的链路预测软件功能模块图

5.3 系统实现

5.3.1 系统介绍模块实现

在系统首页首先对链路预测进行介绍，然后对系统主要支持算法（逻辑回归和 LLSLP）进行展示，并且对本次所采用的 5 个数据集进行具体的介绍，页面展示如图 5.2 所示。在系统首页可以直接选择进行链路预测功能，点击开始预测后会跳转到链路预测界面，但功能的实现首先需要检查 API 是否在线，如果离线则不能使用链路预测功能。

图 5.2 系统首页

系统提供了 API 状态检查功能，访问 /api/status 接口，获取 API 的在线状态以及 GPU 的可用情况。用户能及时了解系统的运行状态，确保在进行模型训练和预测时系统处于正常工作状态。

系统提供了获取 API 状态的接口，该方法通过 GET 请求获取 API 的状态信息。

```
export function getApiStatus() {
  return api.get('/status')}
```

5.3.2 链路预测模块实现

在链路预测时，用户首先需要选择模型，即数据集以及算法模型，然后选择需要预测的节点对，点击开始预测后在页面中展示预测结果，如图 5.3 所示。预测结果中将展示所需要预测节点对的连接概率和预测结果，并将所有预测结果进行分析展示，如图 5.4 所示。设置了导出结果按钮，可以将结果保存为 csv 文件，对结果进行查看。

图 5.3 链路预测界面

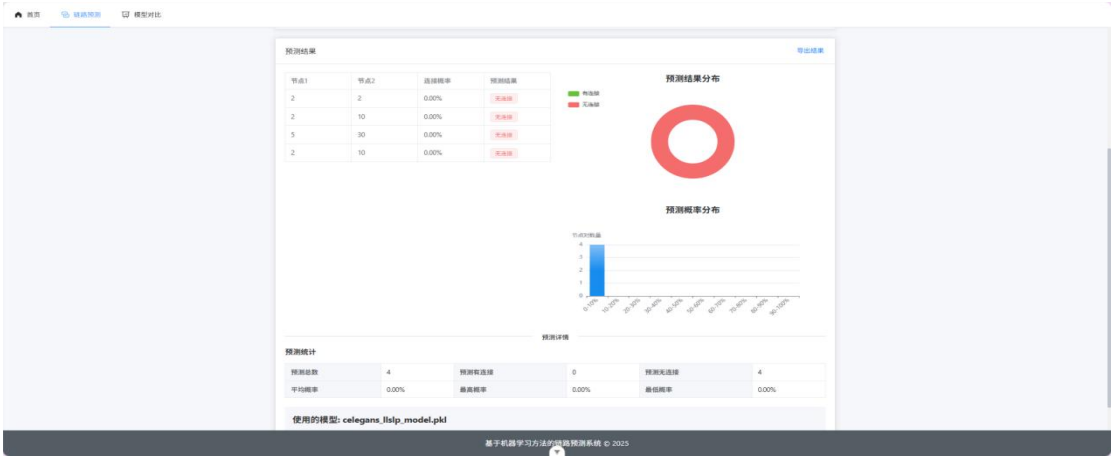


图 5.4 链路预测结果

通过 /api/datasets 接口，系统可以列出所有可用的数据集，方便用户选择合适的数据集进行模型训练和预测。通过 /api/models 接口，系统可以列出所有已训练的模型，方便用户了解模型的基本情况并选择合适的模型进行预测。使用 /api/predict 接口，选择预训练的模型，输入需要预测的节点对。系统加载相应的模型和特征缩放器提取节点对的特征，进行标准化处理后进行预测，预测结果，包括节点对、连接概率和预测结果等信息被返回。系统支持将预测结果和模型对比结果以 CSV 文件的形式导出，方便用户进行进一步的分析和处理。

```
export function getDatasets() {
  return api.get('/datasets')}
export function getModels() {
  return api.get('/models')
}
export function predictLinks(modelName, nodePairs) {
  console.log('准备发送预测请求 - 模型:', modelName)
```

```
  console.log('准备发送预测请求 - 节点对:', JSON.stringify(nodePairs))
  // 确保节点对是普通数组而非响应式对象
  const normalizedPairs = nodePairs.map(pair => {
    if (Array.isArray(pair)) {
      return [Number(pair[0]), Number(pair[1])]
    }
    return pair
  })
  return api.post('/predict', {
    model_name: modelName,
    node_pairs: normalizedPairs
  })
}
```

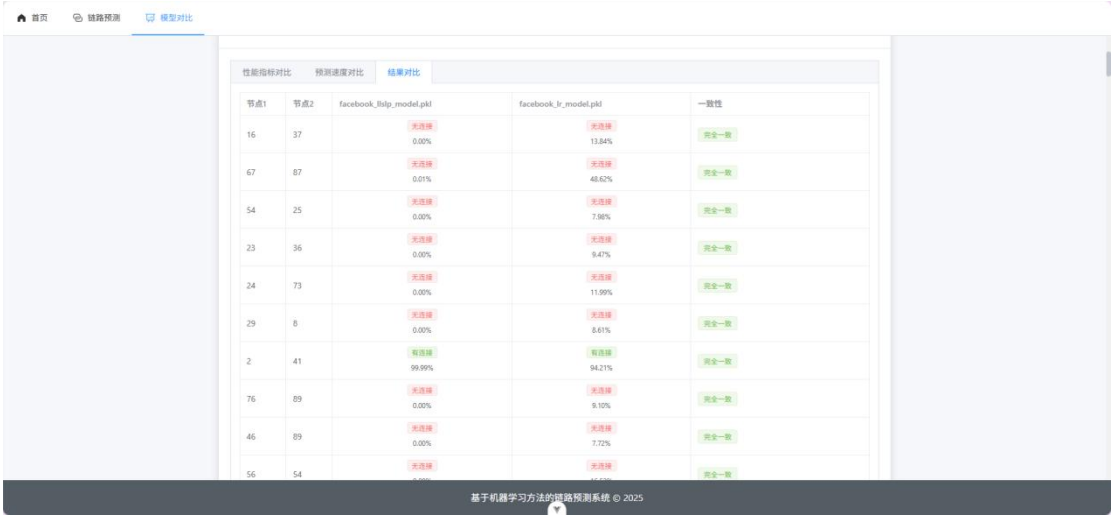
5.3.3 模型对比模块

在模型对比页面，需要选择数据集以及要比较的模型，可以对测试的节点数进行设置，如图 5.5 所示，点击开始比较按钮后，在界面的下方会对模型对比（包括性能指标对比，预测速度对比以及预测结果对比）结果进行直观的展示，如图 5.6 所示。同样，也可以对对比结果进行导出，保存为 csv 文件。

图 5.5 模型对比页面

（a）性能指标对比

（b）预测速度对比



（c）结果对比

图 5.6 模型性能对比

5.3.4 错误消息本地化

为了给用户提供更友好的错误提示，系统实现了错误消息的本地化功能。定义了一个错误消息映射表 errorMessages：

```
const errorMessages = {
  // 数据集相关
  'Dataset directory .* not found': '数据集目录不存在',
  'Cannot find dataset directory matching': '找不到匹配的数据集目录',
  'No valid file found in': '数据集目录中没有有效的数据文件',
  // 模型相关
  'Model file not found': '模型文件不存在',
  'Scaler file not found': '特征缩放器文件不存在',
  'Failed to load graph': '加载图数据失败',
  'No valid node pairs for prediction': '没有有效的节点对用于预测',
  // 请求相关
  'Missing request data': '请求数据缺失',
  'Missing model name or node pairs': '缺少模型名称或节点对数据'
}
```

该映射表将英文的错误消息模式映射为对应的中文错误消息。然后，通过 localizeErrorMessage 函数实现错误消息的本地化：

```
function localizeErrorMessage(error) {
  if (!error || !error.response || !error.response.data || !error.response.data.error) {
    return '未知错误'
  }
  const errorMsg = error.response.data.error
  for (const [pattern, translation] of Object.entries(errorMessages)) {
    const regex = new RegExp(pattern)
    if (regex.test(errorMsg)) {
      return translation
    }
  }
```

```
  return errorMsg
}
```

该函数会检查错误对象及其响应数据，如果匹配到错误消息模式，则返回对应的中文错误消息，否则返回原始错误消息

5.3.5 请求和响应拦截器

为了方便调试，系统添加了请求和响应拦截器。

1. 请求拦截器会在每个请求发送前打印请求的方法、URL 和数据。对于预测请求，还会额外打印请求的详细信息。

```
  config => {
    console.log('API 请求:', config.method.toUpperCase(), config.url, config.data)
    // 对于预测请求，特别检查
    if (config.url === '/predict' && config.method === 'post') {
      console.log('预测请求详情:')
      console.log('- URL:', `${baseURL}${config.url}`)
      console.log('- 请求体:', JSON.stringify(config.data))
    }
    return config
  },
  error => {
    console.error('API 请求错误:', error)
    return Promise.reject(error)
  }
)
```

2. 响应拦截器会在收到响应后打印响应的状态码和请求的 URL。如果响应出现错误，会打印错误信息，并为错误对象添加本地化的错误消息。

```
api.interceptors.response.use(
  response => {
    console.log('API 响应:', response.status, response.config.url)
    return response
```

```
  },
  error => {
    console.error('API 响应错误:',
      error.response ?
        `${error.response.status} ${error.response.config.url}` :
        error.message
    )
    // 添加本地化错误消息
    if (error.response && error.response.data) {
      error.localizedMessage = localizeErrorMessage(error)
    }
    return Promise.reject(error)
  }
)
```

综上所述，本系统通过 axios 实现了与后端 API 的通信，提供了多种 API 请求方法，同时添加了错误消息本地化和请求/响应拦截器，方便开发和调试。

第 6 章 结论与展望

6.1 研究工作总结

本研究在复杂网络中，针对链路预测问题，对比基于机器学习的 LR 算法与 LLSLP 方法进行实验。LR 算法为经典机器学习算法。LLSLP 通过提取节点对间包括共同邻居数、Jaccard 系数、Adamic-Adar 指数、优先连接指标等 8 个相似性指标构建含线性与非线性特征的输入空间，模型架构采用 Stacking 集成学习框架，以逻辑回归（LR）和两个不同参数配置的 LightGBM 作为基模型分别学习特征线性组合关系与高阶交互模式，并通过 LightGBM 元模型融合基模型预测概率实现特征二次利用与预测结果优化。实验在 Zachary_Karate 社交网络、Jazz 音乐家合作网络、Facebook 社交网络等 5 个真实数据集上进行，结果显示 LLSLP 算法在 AUC、准确率、精确率、召回率和 F1 分数等关键指标上优于单一逻辑回归模型，验证集成学习对提升预测精度与泛化能力的有效性。

6.2 研究不足与展望

本文的研究虽然有一定结果但具有局限性。算法当前仅适用于静态同质无权无向网络，缺乏对动态演化网络如随时间变化的社交关系或异质网络如包含多类型节点和边的学术网络的处理能力，且缺少对网络时序特征和异构特征的建模能力；模型采用多基模型集成和 5 折交叉验证机制，在大规模网络如节点数超百万的数据集训练时计算复杂度高、内存占用和运行时间显著增加，限制其在实时场景的应用；研究仅利用网络拓扑结构特征未融合节点属性信息如用户年龄等，难以充分捕捉复杂网络多模态数据的潜在关联。未来研究拟从以下四方面展开：一是引入动态图神经网络或时序卷积网络，结合时间序列分析技术构建动态链路预测模型以适应金融交易网络、交通流量预测等动态场景；二是针对异质网络设计基于元路径的特征提取方法，并改进 Stacking 框架以支持多类型节点和边的建模，如在电商网络中预测用户与商品、商品与品类之间的潜在连接；三是通过注意力机制优化基模型权重分配，或采用知识蒸馏等模型压缩技术降低计算成本，提升算法在大规模数据中的效率；四是融合节点属性、文本描述、图像特征等多模态数据利用 GraphCL[24] 、RGCN[25] 等跨模态嵌入技术生成更具表达力的节点表示，以进一步提升预测精度。这些改进将推动链路预测技术向动态化、异质化、高效化和实用化方向发展并拓展其在社交推荐、生物医学、智能交通等领域的应用场景。

参考文献

[1] 黄雅倩, 王家乐, 杜鑫, 等. 基于加权网络指标集的关键类预测方法[J/OL]. 复杂系统与复杂性科学,1-8[2025-05-22].http://kns.cnki.net/kcms/detail/37.1402.N.20250421.1725.004.html.

[2] 谢鸿博. 前后端分离架构下基于图神经网络的社交网络关系挖掘[J]. 信息技术与信息化, 2024, (12): 104-107.

[3] 郭昕昀. 基于生物网络的磷酸化位点特异激酶预测研究[D]. 南昌大学, 2022.

[4] 郭方平. 以人工智能技术为基础的大数据信息网络探析[J]. 数字通信世界, 2024, (06): 106-108.

[5] 高方宏. 基于交通网络的信息预测方法研究[D]. 上海师范大学, 2023.

[6] 丁娜, 郝江涛. 基于大数据的通讯技术网络中人工智能应用[J]. 中国新通信, 2020, 22(07): 97-98.

[7] 李朋远, 于华, 江成. 中文语境下基于事件关联挖掘的金融网络构建与分析[J]. 中国科学院大学学报, 2021, 38(02): 270-279.

[8] 汤永新, 齐敬英. 基于共同邻居的小度节点有利链路预测算法[J]. 现代电子技术, 2021, 44(05): 37-40.

[9] 孙晶. 分类数据的 Word2Vec 与 Jaccard 相似度聚类方法的比较分析[J]. 软件, 2024, 45(09): 49-51.

[10] 郝佳辉, 万源, 张宇航. 融合位置和结构信息的图神经网络的节点学习研究[J/OL]. 计算机科学, 1-13[2025-05-22].http://kns.cnki.net/kcms/detail/50.1075.TP.20240913.0909.002.html.

[11] 高彧馨. 基于 LR-SMOTE 的谣言传播者意图倾向预测模型[J]. 网络安全技术与应用, 2024, (11): 41-43.

[12] 董梦媛, 郑晓亮. 基于改进的 SVMSMOTE-Lightgbm 煤与瓦斯突出预测[J]. 佳木斯大学学报(自然科学版), 2025, 43(02): 5-8+16.

[13] D Liben-Nowell, J Kleinberg. The Link-Prediction Problem for Social Networks[J]. Am. Soc. Inform. Sci. Technol. 2007,52: 1019.

[14] A Clauset, C Moore, M E J Newman. Hierarchical structure and the prediction of missing links in networks[J]. Nature, 2008, 453: 98.

[15] Menon A, Elkan C. Link prediction via matrix factorization[J]. Proceedings SIAM International Conference on Data Mining, 2011: 1228 - 1239.

[16] 秦红武, 赵猛, 马秀琴, 等. 基于学术水平聚类的科研合作者推荐模型[J]. 计算机工程与应用, 2022, 58(21): 172-181.

[17] Jiahui W, Kun Y, Liang D, et al. An efficient approach for multiple probabilistic inferences with Deepwalk based Bayesian network embedding[J]. Knowledge-Based Systems, 2022, 239.

[18] 杨鹏飞, 王姝祺, 黄嘉阳, 等. 基于特征相似性和杰卡德中值理论的学习路径推荐方法[J]. 计算机科学, 2024, 51(10): 153-161.

[19] 郭静慧, 俞弋峰, 李辉贤. 神经网络算法研究[J]. 软件, 2024, 45(12): 62-64.

[20] 张天舒. 基于子空间聚类的链接预测技术研究[D]. 电子科技大学, 2022.

[21] 李元昊. 复杂网络中的路由传输策略及传播动力学的研究[D]. 北京邮电大学, 2021.

[22] 欧文仪, 王成成, 杨晖. 高职课堂管理社会网络中的优先连接现象研究[J]. 太原城市职业技术学院学报, 2023, (11): 78-81.

[23] 杨硕, 史亚平. 基于 Python+Flask 的在线考试系统设计与实现[J]. 电脑知识与技术, 2025, 21(02): 47-49+56.

[24] Yang X, Yang G, Chu J. GraphCL-DTA: A Graph Contrastive Learning With Molecular Semantics for Drug-Target Binding Affinity Prediction[J]. IEEE journal of biomedical and health informatics, 2024, PP

[25] Jeyaraman P B, Dai T B, Fang Y. Temporal Relational Graph Convolutional Network Approach to Financial Performance Prediction[J]. Machine Learning and Knowledge Extraction, 2024, 6(4): 2303-2320.

致　谢

寒来暑往，几度春秋，行文至此，百感交集。总以为岁月漫长，却未料时光在书页翻动间悄然流转，当键盘敲下“致谢”二字时，恍然惊觉象牙塔的晨钟已至尾声。从大一走到现在，一路跌跌撞撞，或许遗憾、或许不舍，目光所及之处皆是回忆。

一朝沐杏雨，一朝念师恩。首先感谢我的恩师杨词慧老师。大学四年中，杨词慧老师始终像一盏明灯指引我前进的方向，无论是在学业上还是生活上总是为我指点迷津，让我始终在正确的道路上前行。其次要感谢我的指导老师张胜和程世明老师。张胜老师在选题时，结合我以后的规划为我选择最适合的选题；从 0 开始学习，总是给予我耐心的指导和宝贵的意见，严谨的研究态度让我受益匪浅；撰写论文时，陪我字字斟酌，倾尽所能点拨我。程世明老师在做实验的过程中为我提供了较好的实验环境。同时也感谢专业的每一位老师，耐心的教学给我们未来的学习打下坚实的基础。

春晖寸草，山高海深，我要感谢我的家人。无论做什么，他们总是支持我的决定，给予我尊重，尽他们所能帮助我，让我勇敢追梦。

山水一程，三生有幸，感谢我的朋友们。多年以后我可能早已忘记书本上的知识，但是我永远也不会忘记一起熬夜追剧的我们，一起临时抱佛脚准备考试的手忙脚乱，我会记得凌晨 6 点起来跑步的荒谬，也会记得相册里留下的搞怪照片。我们或许在日后会踏上不同的道路，但是我希望每个人都能在自己的领域闪闪发光！

道阻且长，行则将至。最后我要感激一直在路上的自己。大学四年，笑过，哭过，很多个夜晚在想为什么过的这么累，现在回头看，真的很感激那个含泪坚持的自己。

落幕的是我的大学生活，而不是我的人生，这个世界很大，我想活成自己喜欢的样子。

南昌航空大学，感谢相遇，后会有期。